Assignment 17

Q.1 Implement Doubly Linear Linked List Using Head and Tail

Problem Statement

Write a program to implement a **Doubly Linear Linked List** using both **Head** and **Tail** pointers. Perform the following operations:

- AddFirst
- AddLast
- DisplayForward
- DisplayBackward
- DeleteFirst
- DeleteLast
- AddPosition
- DeletePosition

Concepts Used

- Doubly Linked List
- Structures (`struct`)
- Pointers
- Dynamic Memory Allocation (`malloc()` and `free()`)
- Head and Tail Pointers
- Functions
- Traversal
- Menu-Driven Programming
- Conditional Statements (`if`)
- Looping (`while`)

Step-by-Step Approach

AddFirst

1. Create a new node.
2. Store the data in the node.
3. Set the new node's `next` to the current head.
4. Set the current head's `prev` to the new node.
5. Update the head pointer.
6. If the list is empty, update both head and tail.

AddLast

1. Create a new node.
2. Store the data in the node.
3. Set the new node's `prev` to the current tail.
4. Set the current tail's `next` to the new node.
5. Update the tail pointer.
6. If the list is empty, update both head and tail.

DisplayForward

1. Check whether the list is empty.
2. Start traversal from the head.
3. Display each node's data.
4. Continue until `NULL` is reached.

DisplayBackward

1. Check whether the list is empty.
2. Start traversal from the tail.
3. Display each node's data.
4. Continue until `NULL` is reached.

DeleteFirst

1. Check whether the list is empty.
2. Store the head node in a temporary pointer.
3. Move the head to the next node.
4. Set the new head's `prev` to `NULL`.
5. Free the deleted node.
6. If only one node exists, update both head and tail to `NULL`.

DeleteLast

1. Check whether the list is empty.
2. Store the tail node in a temporary pointer.
3. Move the tail to the previous node.
4. Set the new tail's `next` to `NULL`.
5. Free the deleted node.
6. If only one node exists, update both head and tail to `NULL`.

AddPosition

1. Accept the position and data.
2. If the position is `1`, call **AddFirst()**.
3. If the position is the last position, call **AddLast()**.

4. Traverse to the required position.
5. Insert the new node between two existing nodes.
6. Update both `next` and `prev` links.

DeletePosition

1. Accept the position.
2. If the position is `1`, call **DeleteFirst()**.
3. If the position is the last position, call **DeleteLast()**.
4. Traverse to the specified position.
5. Update both `next` and `prev` pointers.
6. Free the deleted node.

Test Cases

Test Case 1 – AddFirst

Input

```
AddFirst(30)
AddFirst(20)
AddFirst(10)
```

Output

```
10 <-> 20 <-> 30 <-> NULL
```

Test Case 2 – AddLast

Input

```
AddLast(10)
AddLast(20)
AddLast(30)
```

Output

```
10 <-> 20 <-> 30 <-> NULL
```

Test Case 3 – DisplayForward

Input

```
10 <-> 20 <-> 30 <-> 40
```

Output

```
10 20 30 40
```

Test Case 4 – DisplayBackward

Input

```
10 <-> 20 <-> 30 <-> 40
```

Output

```
40 30 20 10
```

Test Case 5 – DeleteFirst

Input

```
10 <-> 20 <-> 30 <-> NULL
```

Output

```
20 <-> 30 <-> NULL
```

Test Case 6 – DeleteLast

Input

```
10 <-> 20 <-> 30 <-> NULL
```

Output

```
10 <-> 20 <-> NULL
```

Test Case 7 – AddPosition

Input

```
10 <-> 20 <-> 40

Position = 3
Data = 30
```

Output

```
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

Test Case 8 – DeletePosition

Input

```
10 <-> 20 <-> 30 <-> 40

Position = 3
```

Output

```
10 <-> 20 <-> 40 <-> NULL
```
